

## 파전 파티

보한은 파티를 열려고 한다. 파전을 좋아하기 때문에 파티에 파전을 근처 가게에서 사서 대접하려고 한다. 가게에는 0부터  $N - 1$ 까지 번호가 매겨진  $N$ 가지 맛의 파전이 있다. 보한은 각각의 맛마다  $N$ 장의 파전을 사서, 총  $N^2$  장의 파전을 사려고 한다. 각각의 파전은 봉투에 넣기 전에 정확히  $K$  조각으로 나누어진다. 봉투는 0부터  $N^2 - 1$ 로 번호가 매겨져 있다.  $F[i]$ 가 봉투  $i$ 에 들어있는 파전 맛이라고 하자.

파티가 열리는 동안, 보한은 모든 손님들이 다음과 같은 게임을 하게 한다.

먼저, 보한은  $N$  명의 손님을 고른 후 번호가  $0, 1, \dots, N - 1$ 로 붙어 있는 방에 한 방에 한 명이 들어가게 배정한다. 나머지 손님들은 게임이 끝날때까지 밖에서 기다린다. 모든  $N$  개의 방은 정확히 똑같이 생겼기 때문에, 손님들은 자신이 어느 방에 있는지 알지 못한다.

다음, 각각의 방  $i$ 에 대해서 ( $0 \leq i < N$ ), 보한은 방  $i$ 에 들어가서 이 방에 있는 손님에게 몰래 정수  $P[i]$ 를 알려주는데, 이를 **금지된 맛**이라고 하자. 손님에게 보한이 방 번호를 알려줄 수도 있고 아닐 수도 있다. 보한이 알려주는  $[P[0], P[1], \dots, P[N - 1]]$ 이  $[0, 1, \dots, N - 1]$ 의 순열이라는 것은 보장된다.

다음,  $N$  라운드가 진행된다. 라운드  $i$ 에서, 보한은 봉투  $0, 1, \dots, N^2 - 1$ 을 하나씩 차례로 방  $i - 1$ 로 가져온다. 이 방의 손님이 봉투  $j$ 를 받았을때 다음 두 가지를 알게 된다.

- $F[j]$ , 봉투  $j$ 에 들어있는 파전 맛
- 이 봉투에 남아있는 조각 수

다음 봉투를 받기 전에, 방  $i - 1$ 에 있는 손님은 이 봉투에서 몇 조각을 먹을 지 정해야 한다. 먹을 수 있는 조각의 양은 상황에 따라 다르다.

- 만약  $P[i - 1] \neq F[j]$ 이면 손님은 이 봉투에서 얼마든지 조각을 먹을 수 있다.
- 만약  $P[i - 1] = F[j]$ 이면 손님은 이 봉투에서 조각을 먹을 수 없다.

손님은 순열  $F[0], F[1], \dots, F[N^2 - 1]$ 에 대한 정보를 미리 알지 못한다는데 유의하라.

$N$  라운드가 모두 끝나면, 밖에 있던 손님들에게 봉투의 수열이 주어진다. 봉투를 보면, 이 봉투에 있던 파전이 무슨 맛인지를 알 수 있고 몇 조각이 남아있는지 알 수 있다. 이 정보를 가지고  $P[0], P[1], \dots, P[N - 1]$  값을 알아내야 한다.

손님들은 게임이 시작하기 전에 정보를 주고받을 수 있다. 또  $N, K$  값은 미리 알고 있다. 당신의 목표는 밖에 있는 손님들이 항상 정확한  $P[0], P[1], \dots, P[N - 1]$  값을 알아내는 전략을 구현하는 것이다.

## Implementation details

세 함수를 구현해야 한다.

방  $0, 1, \dots, N - 1$ 에 있는 손님들을 위해 다음 두 함수를 구현해야 한다.

```
void init(int N, int K, int p, int r)
```

손님이 방  $i$ 에 있다고 하자.

- $N$ : 파전 맛 가지수
- $K$ : 게임이 시작하기 전 각 파전을 나눈 조각 수
- $p = P[i]$ : 방  $i$ 에서 먹을 수 없는 금지된 맛
- 만약 보한이 손님에게 어느 방에 있는지를 알려줬다면  $r = i$ 이고 그렇지 않으면  $r = -1$ 이다.

```
int strategy(int b, int f, int s)
```

손님이 방  $i$ 에 있다고 하자.

- $b$ : 현재 봉투 번호
- $f = F[b]$ : 봉투  $b$ 에 들어있는 파전 맛
- $s$ : 봉투  $b$ 에 남아있는 조각 수
- 이 함수는 음이 아닌 정수  $x$ 를 리턴해야 하는데, 이 값은 손님이 먹기로 한 조각 수이다.
  - 만약  $f \neq P[i]$ 이면,  $0 \leq x \leq s$ 이어야 한다.
  - 만약  $f = P[i]$ 이면,  $x = 0$ 이어야 한다.

밖에 있는 손님들을 위해서 다음 함수를 구현해야 한다.

```
std::vector<int> guess(int N, int K, std::vector<int> F,  
                      std::vector<int> S)
```

- $N$ : 파전 맛 가지수
- $K$ : 게임이 시작하기 전 각 파전을 나눈 조각 수
- $F$ : 길이  $N^2$ 인 배열로  $F[i]$ 는 봉투  $i$ 에 있는 파전 맛
- $S$ : 길이  $N^2$ 인 배열로  $S[i]$ 는  $N$  라운드를 돈 다음 봉투  $i$ 에 남아 있는 조각 수
- 이 함수는 길이  $N$ 인 배열  $P$ 를 리턴해야 하는데,  $P[i]$ 는 방  $i$ 에 있는 손님에게 알려준 수

각 테스트 케이스는  $T$ 개의 독립적인 게임을 포함한다.

채점 프로세스에서,  $T$ 개 게임 각각마다  $N + 1$ 개의 프로세스가 동작할 것이다.  $0 \leq i < N$ 인 각  $i$ 에 대해서, 프로세스  $i$ 는 방  $i$ 에 있는 손님을 나타낸다. 프로세스  $N$ 은 밖에 있는 손님들을 나타낸다.  $0 \leq i < N$ 인 각  $i$ 에 대해서, 프로세스  $i$ 가 끝나야 프로세스  $(i + 1)$ 가 시작한다.

프로세스  $0, \dots, N - 1$ :

- `init` 이 정확히 한 번 호출된다.
- `init` 다음에 `strategy`이  $N^2$ 번 호출된다.
  - $j$ 번째 호출에서  $b = j - 1$ 임은 보장된다.

프로세스  $N$ :

- `guess`이 정확히 한 번 호출된다.

채점 시스템은 다른 게임들의 프로세스를 서로 엇갈려서 수행할 수 있지만, 각각의 게임에 대한 프로세스의 순서가 지켜진다는 것은 보장된다.

## Constraints

- $1 \leq T \leq 400$ .
- $2 \leq N \leq 30$ .
- 한 테스트 케이스에서 모든 게임에 대한  $N^3$ 의 합은 27 000 이하.
- $1 \leq K \leq N$
- $K \in \{1, 3, N\}$ .
- $[P[0], P[1], \dots, P[N - 1]]$ 은  $[0, 1, \dots, N - 1]$ 의 순열이다.
- $0 \leq j < N^2$ 인 각각의  $j$ 에 대해  $0 \leq F[j] < N$ .
- $0 \leq i < N$ 인 각각의  $i$ 는  $[F[0], F[1], \dots, F[N^2 - 1]]$ 에서 정확히  $N$ 번 나온다.
- 채점 시스템은 **적응적이지 않다**. 즉,  $[P[0], P[1], \dots, P[N - 1]]$ 와  $[F[0], F[1], \dots, F[N^2 - 1]]$ 는 `init`를 처음 호출하기 전에 이미 정해져 있다.

## Subtasks

| 서브태스크 | 점수 | 추가적인 제약 조건                                                                                                                              |
|-------|----|-----------------------------------------------------------------------------------------------------------------------------------------|
| 1     | 8  | $K = 1, N = 2$ .                                                                                                                        |
| 2     | 7  | $K = 1$ 이고 보한은 모두에게 방 번호를 알려준다.                                                                                                         |
| 3     | 14 | $K = 1$ 이고 $0 \leq i < N$ 인 각 $i$ 에 대해 $[F[i \cdot N], F[i \cdot N + 1], \dots, F[i \cdot N + N - 1]]$ 는 $[0, 1, \dots, N - 1]$ 의 순열이다. |
| 4     | 18 | $K = N$ .                                                                                                                               |
| 5     | 21 | $K = 3, N \geq 3$ .                                                                                                                     |
| 6     | 32 | $K = 1$ .                                                                                                                               |

## Example

테스트 케이스에  $T = 1$ 개의 게임이 포함되고  $N = 2, K = 2, P = [1, 0], F = [1, 0, 0, 1]$ 인 경우를 고려해보자.

$S[i]$ 가 현재 봉투  $i$ 에 들어있는 조각수라고 하자. 처음에는  $S = [2, 2, 2, 2]$ 이다.

손님들이 다음과 같은 전략을 게임 시작 전에 세웠다고 하자.

- 방에 있는 손님들은 만약 주어진 파전을 먹을 수 있다면 다음과 같이 행동한다.
  - 만약 주어진 파전이 먹을 수 있는 파전 중 처음으로 주어진 것이라면, 남은 조각을 다 먹는다.
  - 그렇지 않다면, 한 조각만 먹는다.
- 밖에 있는 손님들이 다음과 같은 전략을 게임 시작 전에 세웠다고 하자.
  - 만약  $S = [0, 0, 1, 1]$ 이라면  $P = [1, 0]$ 라고 추측한다.
  - 그렇지 않다면,  $P = [0, 1]$ 라고 추측한다.

게임은 다음과 같이 진행된다.

먼저, 채점 시스템은 방 0에 있는 손님을 나타내는 프로세스 0을 시작하고 `init(2, 2, 1, -1)`을 호출한다. 초기화가 끝나면, 채점 시스템은 다음과 같은 호출을 한다.

| 함수 호출             | 리턴값 |
|-------------------|-----|
| strategy(0, 1, 2) | 0   |
| strategy(1, 0, 2) | 2   |
| strategy(2, 0, 2) | 1   |
| strategy(3, 1, 2) | 0   |

$P[0] = F[0] = F[3] = 1$ 이기 때문에 첫번째와 네번째 호출의 리턴값은 0이어야 한다.

이 프로세스가 끝나면 (1라운드가 끝나면)  $S = [2, 0, 1, 2]$ 이다.

다음, 채점 시스템은 방 1에 있는 손님을 나타내는 프로세스 1을 시작하고 `init(2, 2, 0, -1)`을 호출한다. 초기화가 끝나면, 채점 시스템은 다음과 같은 호출을 한다.

| 함수 호출             | 리턴값 |
|-------------------|-----|
| strategy(0, 1, 2) | 2   |
| strategy(1, 0, 0) | 0   |
| strategy(2, 0, 1) | 0   |
| strategy(3, 1, 2) | 1   |

$P[1] = F[1] = F[2] = 0$ 이므로 두번째와 세번째 호출의 리턴값은 0이어야 한다.

이 프로세스가 끝나면 (2라운드가 끝나면)  $S = [0, 0, 1, 1]$ 이다.

마지막으로, 채점 시스템은 밖에 있는 손님을 나타내는 프로세스 2를 시작한다. 채점 시스템은 다음과 같은 호출을 한다.

| 함수 호출                                   | 리턴값    |
|-----------------------------------------|--------|
| guess(2, 2, [1, 0, 0, 1], [0, 0, 1, 1]) | [1, 0] |

밖의 손님들은 답을 맞추었으므로, 이 테스트 케이스는 정답으로 인정된다.

밖의 손님이 위의 전략을 이용해서 항상 정답을 맞출 수 있는 것은 아니라는데 유의하라.

이 문제에 첨부된 패키지에는 이 예제에 추가로 다른 예제 입력이 포함되어 있다.

## Sample Grader

샘플 그레이더는 **테스트 케이스당 게임 하나만** 지원한다.

입력 양식:

```
N K
P[0] P[1] ... P[N-1]
F[0] F[1] ... F[N*N-1]
reveal
```

- `reveal`은 0 또는 1이다.
  - 만약 `reveal`이 0이면, 그레이더는 보한이 아무에게도 방 번호를 알려주지 않은 경우로 동작한다.
  - 만약 `reveal`이 1이면, 그레이더는 보한이 모두에게 방 번호를 알려준 경우로 동작한다.

만약 `guess`가 리턴한 배열이  $[P[0], P[1], \dots, P[N - 1]]$ 와 일치한다면, 샘플 그레이더는 다음을 출력한다.

```
Accepted
```

추가로, 그레이더는 디버깅 용으로 함수 호출을 차례로 출력한다.